



Design and Analysis of Algorithms

Vandana M L

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Asymptotic Notations

Slides courtesy of **Anany Levitin**

Vandana M L

Department of Computer Science & Engineering

Orders of growth of an algorithm's basic operation count is important

How do we compare order of growth??

Using Asymptotic Notations

A way of comparing functions that ignores constant factors and small input sizes

$O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$

$\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$

$\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$

$o(g(n))$: class of functions $f(n)$ that grow at slower rate than $g(n)$

$\omega(g(n))$: class of functions $f(n)$ that grow at faster rate than $g(n)$

O-notation

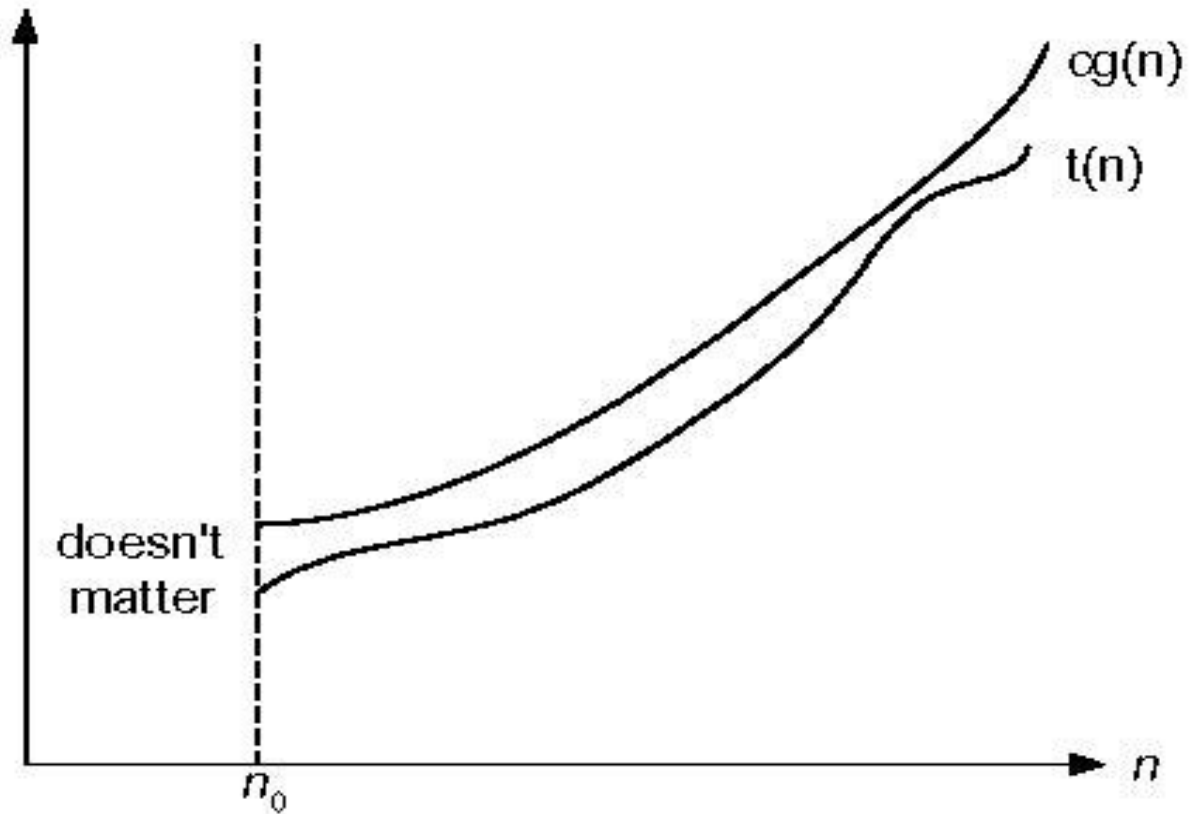


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

O-notation

Formal definition

A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist **some** positive constant c and some nonnegative integer n_0 such that

$$t(n) \leq cg(n) \text{ for all } n \geq n_0$$

Exercises: prove the following using the above definition

$$10n^2 \in O(n^2)$$

$$10n^2 + 2n \in O(n^2)$$

$$100n + 5 \in O(n^2)$$

$$5n+20 \in O(n)$$

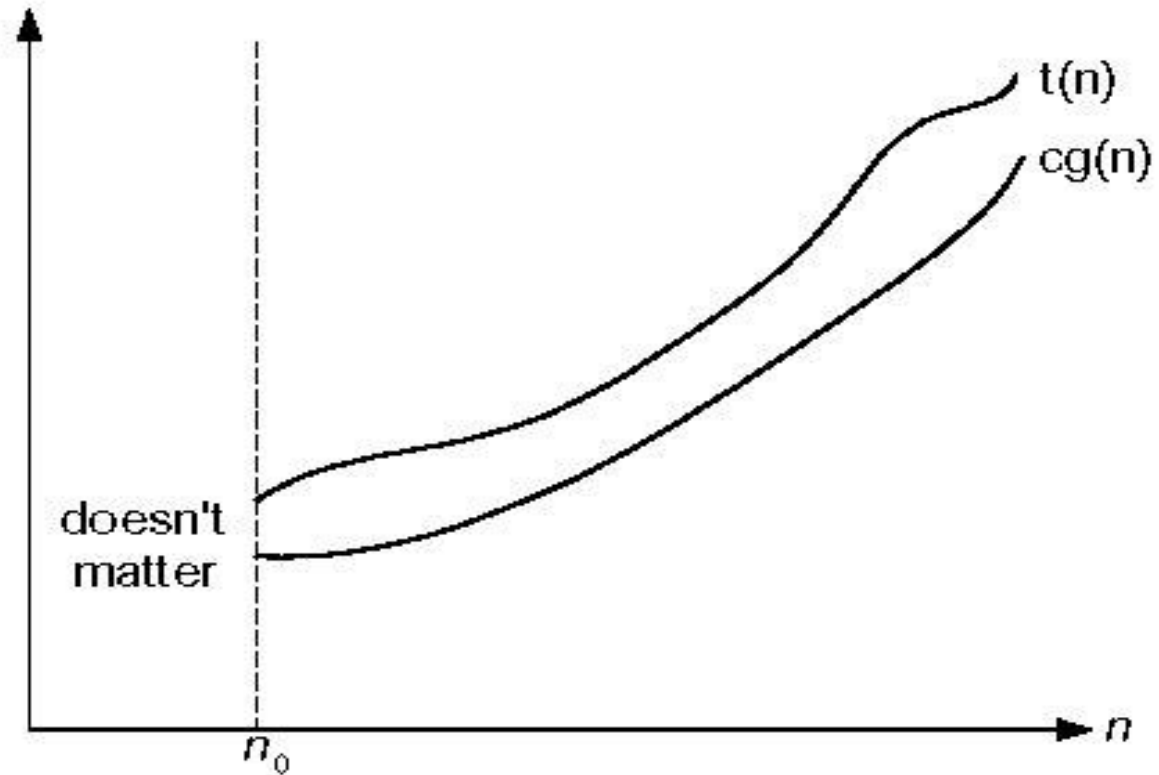


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$

Formal definition

A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some constant multiple of $g(n)$ for all large n ,

i.e., if there exist **some** positive constant c and some nonnegative integer n_0 such that

$$t(n) \geq cg(n) \text{ for all } n \geq n_0$$

Exercises: prove the following using the above definition

$$10n^2 \in \Omega(n^2)$$

$$10n^2 + 2n \in \Omega(n^2)$$

$$10n^3 \in \Omega(n^2)$$

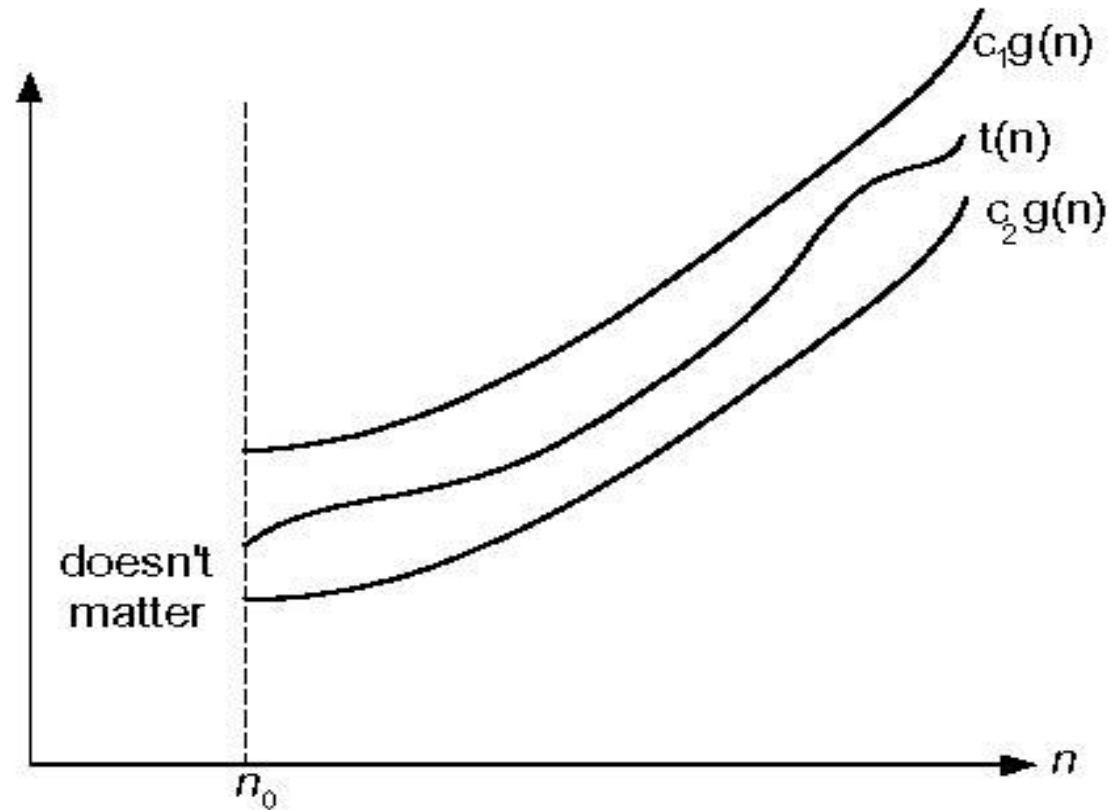


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

Θ -notation

Formal definition

A function $t(n)$ is said to be in $\Theta(g(n))$, denoted $t(n) \in \Theta(g(n))$, if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n ,

i.e., if there exist some positive constant c_1 and c_2 and some nonnegative integer n_0 such that

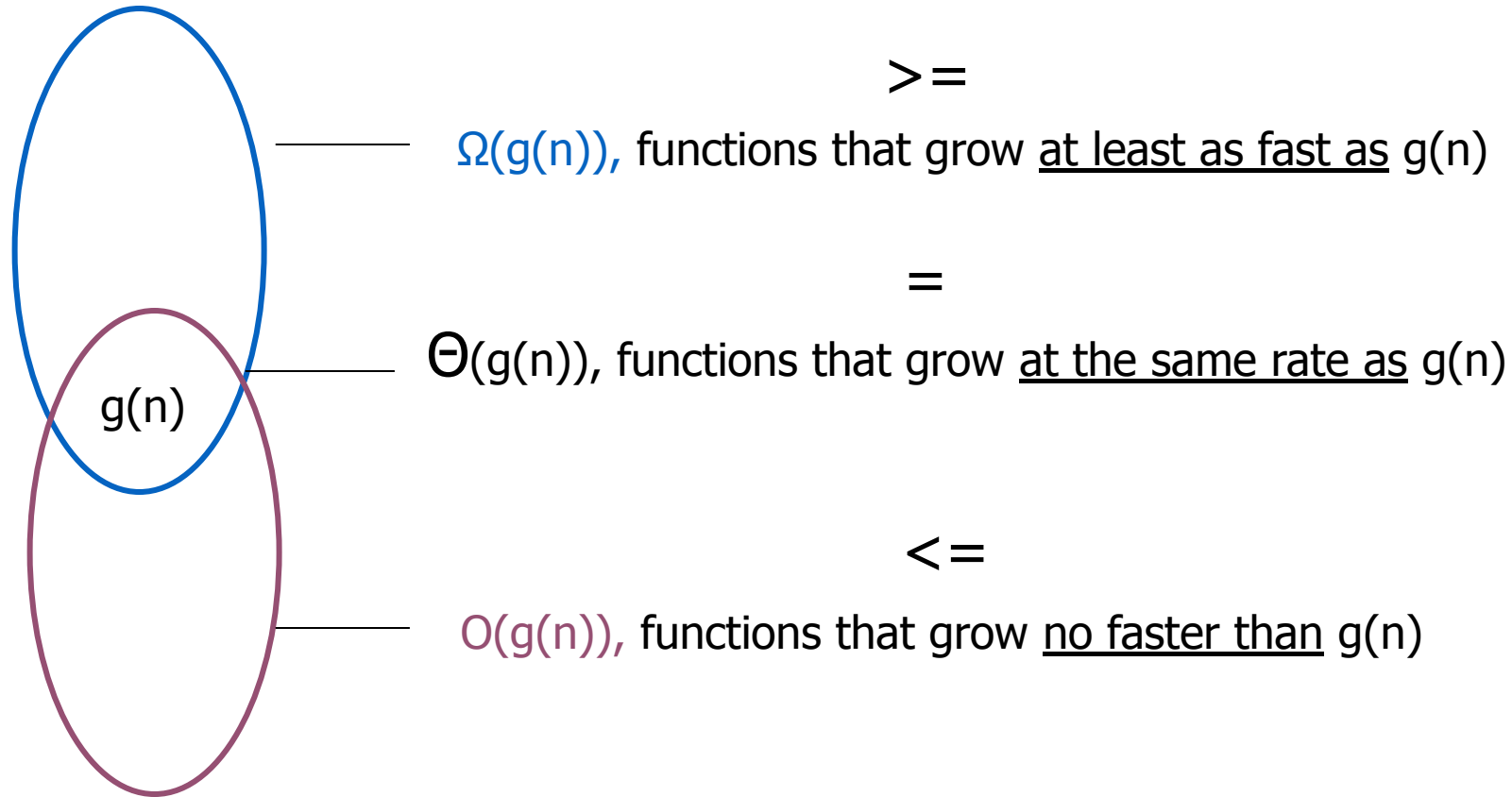
$$c_2 g(n) \leq t(n) \leq c_1 g(n) \text{ for all } n \geq n_0$$

Exercises: prove the following using the above definition

$$10n^2 \in \Theta(n^2)$$

$$10n^2 + 2n \in \Theta(n^2)$$

$$(1/2)n(n-1) \in \Theta(n^2)$$



Little-o Notation

Formal Definition:

A function $t(n)$ is said to be in Little-o($g(n)$), denoted $t(n) \in o(g(n))$, if for any positive constant c and some nonnegative integer n_0

$$0 \leq t(n) < cg(n) \text{ for all } n \geq n_0$$

Little-o Notation

Example: $f(n) = 2n^2$ and $g(n) = n^2$ and $c = 2$

$f(n) = O(g(n))$ - Big-O

$f(n) \neq o(g(n))$ - little-o

If $f(n) = 2n$ & $g(n) = n^2$,
then for any value of $c > 0$,
 $f(n) < c(n^2)$
 $f(n) \neq o(g(n))$

Note : For non-negative functions, $f(n)$ and $g(n)$,
 $f(n)$ is little o of $g(n)$, if and only if,

$f(n) = O(g(n))$ and $f(n) \neq \theta(g(n))$
[strict upper bound, no lower bound]

Little Omega Notation

Formal Definition:

A function $t(n)$ is said to be in Little- $\omega(g(n))$, denoted $t(n) \in \omega(g(n))$,
if for any positive constant c and some nonnegative integer n_0
 $t(n) > cg(n) \geq 0$ for all $n \geq n_0$

Little Omega Notation

Example : If $f(n) = 3n^2 + 2$, $g(n) = n$
then for any value of $c > 0$
 $f(n) > cg(n)$

$$f(n) = \omega(n)$$

Note : For non-negative functions, $f(n)$ and $g(n)$,
 $f(n)$ is little ω of $g(n)$, if and only if

$$f(n) = \Omega(g(n)) \text{ and } f(n) \neq \theta(g(n))$$

[strict lower bound, no upper bound]

Theorems

- If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then
 $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$.
For example,
 $5n^2 + 3n\log n \in O(n^2)$
- If $t_1(n) \in \Theta(g_1(n))$ and $t_2(n) \in \Theta(g_2(n))$, then
 $t_1(n) + t_2(n) \in \Theta(\max\{g_1(n), g_2(n)\})$
- $t_1(n) \in \Omega(g_1(n))$ and $t_2(n) \in \Omega(g_2(n))$, then
 $t_1(n) + t_2(n) \in \Omega(\max\{g_1(n), g_2(n)\})$

Implication: The algorithm's overall efficiency will be determined by the part with a larger order of growth, i.e., its least efficient part.

Why does asymptotic notation, such as Big-O, deliberately omit constant factors when comparing algorithm growth rates?

- A. Because constant multipliers are irrelevant for any practical input size
- B. Because these factors are influenced by hardware, software, and coding specifics, and we want a machine-independent analysis of algorithmic scaling
- C. Because including constants would invalidate the mathematical foundations of asymptotic comparison
- D. Because modern compilers automatically remove constant factors during optimization

Answer: B. Because these factors are influenced by hardware, software, and coding specifics, and we want a machine-independent analysis of algorithmic scaling

Which of the following conditions **correctly defines** when a function $t(n)$ belongs to $O(g(n))$?

- A. $t(n) < g(n)$ for all values of n
- B. $t(n) \leq c g(n)$ for infinitely many values of n
- C. $t(n) \leq c g(n)$ for all $n \geq n_0$ for some positive constants c and n_0
- D. $t(n) = c g(n)$ for all sufficiently large values of n

Answer : **C.** $t(n) \leq c g(n)$ for all $n \geq n_0$ for some positive constants c and n_0

Consider $f(n)$ and $g(n)$ as positive-valued functions.

If $f(n)$ is $O(g(n))$ and also $\Omega(g(n))$, what can be concluded about their asymptotic relationship?

- A. $f(n)$ is asymptotically smaller than $g(n)$
- B. $f(n)$ is asymptotically larger than $g(n)$
- C. $f(n)$ grows at the same asymptotic rate as $g(n)$
- D. The given information is insufficient to decide

Answer: C

How does little-o notation differ from big-O notation?

- A. Little-o provides an average-case bound.
 - B. Little-o describes a strict upper bound, while big-O allows equal growth up to constants.
 - C. Little-o describes both upper and lower bounds.
 - D. Little-o ignores lower-order terms, while big-O does not.
-
- B. Little-o describes a strict upper bound, while big-O allows equal growth up to constants.

Which of the following best describes why algorithm analysts prefer Θ -notation over Big-O notation when both can be used?

- A. It only describes the best-case scenario.
- B. It guarantees the exact running time in seconds.
- C. It provides both an asymptotic upper and lower bound, giving a tighter characterization.
- D. It ignores constants and smaller terms, making it simpler.

Answer: C. . It provides both an asymptotic upper and lower bound, giving a tighter characterization

Multiple Choice Questions

Let $t_1(n) = 5n \log n + 3n$ and $t_2(n) = 2n^2 + 7$. According to the theorem $t_1(n) + t_2(n) \in O(\max(g_1(n), g_2(n)))$ if $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, which of the following is the tightest correct application of this rule for $t_1(n) + t_2(n)$?

- A. $O(n^2)$
- B. $O(n^2 \log n)$
- C. $O(n \log n)$
- D. $O(n^3)$

Answer: A. $O(n^2)$



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu